# Week 4- Sorting Algorithms I

Instructors: Dr. Fatima Adamu-Fika
and Ms. Onyinye Okpoko

# Overview

- Sorting is the permuting (rearranging or reordering) of a sequence of objects (elements, also items) to put them in some sort of logical order, i.e., rearranging the elements of an input object (usually an array) according to a well-defined ordering rule. Sorting is a fundamental operation in computing because many programs use it as an intermediate step.

# Categories of sorting

There are two categories of sorting based on whether the objects that need to be sorted can fit into the available memory all at once or not.

- Internal sorting: When all elements that need to be sorted are placed in memory at the same time, then the sorting done is internal.
- External sorting: When all elements that need to be sorted cannot fit in memory all at once, then the sorting is done in phases. The sorted data are put back together. Such sorting is called external sorting. External sorting is used for a massive amount of data, and some sort of external storage is used to hold the large data and the sorted output files.

# Some sorting terminologies

- A **sorting algorithm** is an algorithm that places elements of a list into order, i.e., sorts the elements of a list. The most commonly used orders are numerical order and lexicographical order, both can either be non-decreasing or non-increasing.
- A **sorting problem** is a computational problem that is solved by a sorting algorithm.
- An **in-place sorting algorithm** is a sorting algorithm that does not require an auxiliary data structure when sorting. It, however, might require a small amount of additional storage space for auxiliary variables. It requires no more than a constant amount of additional memory beyond the input itself as it sorts the elements of a list. An in-place algorithm sorts the list by swapping or replacing the elements. An algorithm which does not sort in place is sometimes called a not-in-place or out-of-place algorithm.
- A **stable sorting algorithm** maintains the relative order of elements with equal values. This means equal elements appear in the same order in the sorted output as they do in the input.

# Common sorting algorithms

We will be studying six common sorting algorithms:

1. The three elementary sorting algorithms: Selection sort; Bubble sort and Insertion sort
2. Merge sort
3. Quicksort
4. Heapsort

# Selection sort

Selection sort uses a brute-force approach to sort a given list. Selection sort is a sorting algorithm that selects the smallest element from an unsorted list in each iteration and places that element at the beginning of the unsorted sublist.

# Selection sort algorithm

Algorithm SelectionSort(A)
Input: an array, A[0 … n – 1], with n order-able items.
Output: The array, A, sorted in non-decreasing order.
START

```
1.    for( i = 0; i < n - 1; i++ ) //for i = 0 to n – 2
2.        min = i
3.        for( j = i + 1; j < n; j++ ) //for j = i + 1 to n – 1 \
4.            f( A[j] < A[min]
5.            min = j
6.        Swap( A, i, min )   //swap A[min] with A[i]
7.    return A
```

END

The selection sort is used when:

- a small list is to be sorted;
- cost of swapping does not matter;
- checking of all the elements is compulsory;
- cost of writing to memory matters like in flash memory (since the number of writes/swaps is $O(n)$).

# Bubble sort

- The bubble sort is another brute force approach to sorting order-able lists. The bubble sort compares adjacent elements of the list and swaps them if they are out of order. By repeatedly doing so, we end up ‘bubbling up’ the largest element to its final position in the list.

# Bubble sort algorithm

Algorithm BubbleSort(A)
Input: an array, A[0 … n – 1], with n order-able items.
Output: The array, A, sorted in non-decreasing order.
START
1. for( i = 0; i < n - 1; i++ ) //for i = 0 to n – 2
2. for( j = 0; j < n – 1 - i; j++ ) //for j = 0 to n – 2 – I
3. if( A[j + 1] < A[j] )
4. Swap( A, j, j + 1 ) //swap A[j+1] with A[j]
5. return A
END

The bubble sort is used when:

- complexity does not matter;
- short and simple code is preferred.

# Insertion sort

Insertion sort is a sorting algorithm that places an unsorted element at its suitable place in each iteration. Insertion sort applies the decrease and conquer approach to sort a list. Specifically, it uses the decrease-by-one technique, it assumes a smaller problem of the list has already been solved, leveraging this solution to the smaller problem, for the next unsorted element, it finds its appropriate place among the sorted elements, and it inserts it there.

# Insertion sort algorithm

Algorithm InsertionSort(A)
Input: an array, A[0 … n – 1], with n order-able items.
Output: The array, A, sorted in non-decreasing order.
START

```
1.  for( i = 1; i < n; i++ ) //for i = 1 to n – 1
2.      key = A[i]
3.       j = i – 1
4.      while( j >= 0 && A[j] > key)
5.          A[j + 1] = A[j]
6.          j = j – 1
7.      A[j + 1] = key
8.  return A
```

END

Insertion Sort is used when:

- the array has a small number of elements;
- there are only a few elements left to be sorted.

# Summary of complexities of the elementary sorting algorithms

| | | Bubble sort | Selection sort | Insertion sort |
|---|---|---|---|---|
| Time complexity | Overall | $O(n^2)$ | $\Theta(n^2)$ | $O(n^2)$ |
| | Best | $\Theta(n)$ | $\Theta(n^2)$ | $\Theta(n)$ |
| | Average | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| | Worst | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| Swaps or Movements | Best | $\Theta(1)$ | $\Theta(1)$ | $\Theta(1)$ |
| | Worst | $\Theta(n^2)$ | $\Theta(n)$ | $\Theta(n^2)$ |
| Auxilliary space | | $O(1)$ | $O(1)$ | $O(1)$ |
| Algorithmic paradigm | | Brute Force | | Decrease and conquer |
| Sorting method | | Swapping | Swapping | Shifting |
| Sorting in place? | | Yes | Yes | Yes |
| Stability | | Stable | Not stable | Stable |